

CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

Project Overview

PyPSA-Eur Romania is a comprehensive energy system modeling project combining:

- **Core Modeling:** PyPSA-based optimization using Snakemake workflows for energy system simulations
- **Scenario Management:** Python-based scenario builder and configuration system
- **Interactive Visualization:** Next.js/React web dashboard (in `vizualizer/` folder) for scenario management and results visualization
- **Analysis & Reporting:** Post-processing and stress-test analysis tools
- **Documentation:** Obsidian vault (in `vault/`) containing architecture, planning, and configuration guides

Architecture Overview

High-Level Flow

```
Configuration (YAML)
  ↓
Snakemake Workflow (rules/, scripts/)
  ↓
Network Optimization & Solving
  ↓
Results (networks/, CSVs, figures)
  ↓
Analysis & Reporting (personal_analysis/)
  ↓
Visualization & Dashboard (vizualizer/ web UI or personal_dashboard/ Tkinter)
```

Directory Organization

Core PyPSA-Eur Directories:

- `config/` - YAML scenario configurations (base settings + year-specific templates in `personal_docs/`)
- `scripts/` - Core PyPSA-Eur execution scripts (data prep, network building, optimization)
- `rules/` - Snakemake workflow rule definitions
- `data/` - Raw/processed datasets (weather cutouts, power plant data, geographic boundaries)
- `resources/` - Intermediate processed files from Snakemake rules
- `results/` - Output networks (.nc files), CSVs, and figures

Custom Romania Project Directories:

- [vizualizer/](#) - **Primary web dashboard** (Next.js/React, scenario builder + run queue + results viewer)
- [personal_dashboard/](#) - Legacy Tkinter UI (scenario manager, results viewer); stable but superseded by vizualizer
- [personal_runners/](#) - Scripts to execute baseline/stress scenarios without Snakemake CLI
- [personal_analysis/](#) - Post-processing: CSV generation, reporting, figure creation
- [personal_diagnostics/](#) - Validation tools (config checks, data integrity, URL testing)
- [personal_data_download/](#) - ERA5 cutout and Zenodo dataset acquisition
- [personal_docs/](#) - Implementation documentation and scenario templates
- [vault/](#) - **Obsidian knowledge base** with architecture, running guides, configuration explanations (EN/RO)

Scenario Manager System

The scenario manager logic is shared between the web dashboard ([vizualizer/](#)) and the legacy Tkinter UI ([personal_dashboard/scenario_manager/](#)). Both use:

- **Immutable Template Pattern:** Read-only canonical YAML templates in [personal_docs/](#)
- **Year-Specific Templates:** [scenario_template.yaml](#) (default/2020) and [scenario_template_2023.yaml](#) auto-selected based on cutout year
- **Config Builder:** Generates YAML by merging user inputs with template defaults
- **Dual Editing Modes:** Structured form controls OR raw YAML editor (both update same config)
- **Queue-Based Execution:** Non-blocking subprocess management; status persisted to JSON

Legacy Tkinter key modules ([personal_dashboard/scenario_manager/](#)):

- [config_builder.py](#) - [resolve_template_path\(\)](#) intelligently selects year-specific templates; [build_configs\(\)](#) generates paired baseline+stress YAMLs
- [run_manager.py](#) - Queue management, subprocess orchestration, job state tracking
- [results_index.py](#) - Scans [results/](#) for new-format comparison outputs; validates required CSV presence
- [state_store.py](#) - Persistent JSON state for jobs, history, language preference

Vizualizer – Web Dashboard (Next.js)

Location: [vizualizer/](#) (intentional spelling with 'z')

Stack: Next.js 16.2.4 (App Router), React 19.2.4, TypeScript 5, Tailwind CSS 4, papaparse, yaml

Status: Production-ready — fully replaces the Tkinter UI for day-to-day use.

Pages:

- **Scenario Builder** — form + YAML editor to configure and enqueue paired or single runs
- **Runs** — queue list with live progress, log viewer, cancel support
- **Results** — auto-scanned list of valid comparison outputs; tabs for Summary, CSV Data, Figures, Assumptions

Development:

```
cd vizualizer
npm install      # Install dependencies (already in node_modules)
npm run dev      # Start dev server (http://localhost:3000)
npm run build    # Production build
npm run lint     # ESLint check
```

Runtime detection: The app auto-discovers the conda environment at startup — priority order:

PLANUI_CONDA_PREFIX env var → CONDA_PREFIX → PLANUI_CONDA_ENV → CONDA_DEFAULT_ENV → pypsa → pypsa-eur → system Python.

State persistence: vizualizer/.data/planui-state.json (jobs survive restart; running → interrupted on restart)

Log files: logs/planui-web/<jobId>.log

Known environment issue (Python 3.13 + linopy):

linopy imports google-cloud-storage at module level. The old version 1.31.2 uses the deprecated pkg_resources API, which is missing in Python 3.13. Fix:

```
conda run -n pypsa pip install "google-cloud-storage>=2.10"
```

Full API and architecture details: vault/Vizualizer.md

Key Concepts & Patterns

Configuration Architecture

- **Baseline Configs:** No shocks applied, stress_test.enable=false
- **Stress Configs:** Shock parameters defined, stress_test.enable=true
- **Paired Scenarios:** Typically run baseline + stress together for comparison
- **Geographic Scope:** Romania stress tests typically cover [RO, BG, HU, RS] (shocks applied only to RO)
- **Time Windows:** Specified in UTC; conversions to local EET handled in reporting

Stress Test Parameters

Defined in stress_test YAML block (only in stress configs):

- **Timeseries Shocks:** Load multiplier, hydro availability reduction, gas capacity reduction for specific hours
- **SCADA Proxy:** Ramp rate constraints on controllable generators (hour-dependent limits)
- **Import Caps:** Directional constraints on border interconnectors (time-window dependent)
- **Shock Application:** Pre-solve (in scripts/romania_winter_stress.py) via apply_timeseries_shocks(), then constraint addition via add_scada_proxy_constraints() and add_import_cap_constraints()

Results Format

New-format results (required for both dashboards):

- **Required CSVs:** `system_cost_comparison.csv`, `generation_mix_mwh.csv`, `imp_summary_ro.csv`, `ens_summary.csv`, `curtailment_mwh.csv`, `daily_net_imports_mwh.csv`, `interconnector_flow_congestion.csv`
- **Figures:** PNG + PDF versions (fig_01 through fig_05)
- **Assumptions Note:** `assumptions_limitations.md` documenting shock formulation, constraints, and interpretation limits
- **Storage:** Results organized by comparison name in `results/<output_name>/`

Data Validation & Diagnostics

- `check_romania.py` - Validates scenario configs, cutout references, and network requirements
- `check_csv.py` - Validates output CSV structure and data quality
- `check_url.py` - Verifies external data source availability (ERA5, Zenodo)

Documentation References

For architecture, planning, and configuration details:

- `vault/Index.md` - Master index to all Obsidian documentation
- `vault/Architecture.md` - Stress test implementation plan, shock logic, reporting specs
- `vault/FolderStructure.md` - Directory roles and organization
- `vault/Vizualizer.md` - Web dashboard architecture, API endpoints, env vars, known issues
- `personal_docs/PLAN.md` - Original project scope and acceptance criteria
- `personal_docs/TEMPLATE_ARCHITECTURE.md` - Year-specific template selection system (400+ lines)
- `personal_docs/romania_config_explanation.md` - YAML configuration guide (English)
- `personal_docs/romania_config_explanation_ro.md` - YAML configuration guide (Romanian)
- `personal_docs/PROJECT_ORGANIZATION.md` - Updated folder structure guide with quick-start workflows

For dashboard/UI details:

- `personal_docs/planui.md` - Legacy Tkinter Scenario Manager UI architecture and implementation notes
- `personal_dashboard/README.md` - Tkinter dashboard usage guide

Common Development Tasks

Running Scenario Simulations

Full workflow (recommended for most use cases):

```
cd personal_runners
python run_romania_winter_stress.py          # Baseline + stress with
```

```
comparison
```

This runs both baseline and stress scenarios, generates comparison reports, and validates outputs automatically.

Quick baseline test (for validation):

```
python run_baseline_only.bat # Quick baseline test
```

Use this to validate that the environment is working and network solves correctly without stress shocks.

All seasonal scenarios:

```
python run_all_scenarios.py # All seasonal scenarios (spring, summer, autumn, winter, december)
```

Generates a full suite of baseline + stress pairs across all seasons. Runs sequentially; takes several hours.

Tips:

- Use the web dashboard ([vizualizer](#)) for interactive scenario building and monitoring
- Check logs in [logs/planui-web/](#) for job-specific output
- Results must contain all 7 required CSVs to appear in the Results tab (see below)

Validating Environment and Configuration

Before running scenarios, always validate:

```
cd personal_diagnostics
python check_romania.py # Validate scenario configs and dependencies
python check_csv.py # Validate result output structure and data quality
python check_url.py # Verify external data sources are reachable (ERA5, Zenodo)
```

What each check does:

- [check_romania.py](#) - Verifies YAML config syntax, cutout file references, network existence, and solver availability
- [check_csv.py](#) - Validates that result CSVs match expected schema and contain valid numeric data
- [check_url.py](#) - Tests connectivity to ERA5 MARS API and Zenodo dataset endpoints

Common issues and fixes:

- `FileNotFoundError: cutout not found` → Run `personal_data_download/download_cutout.py` to fetch weather data
- `ModuleNotFoundError: pkg_resources` → Install fix: `conda run -n pypsa pip install "google-cloud-storage>=2.10"`
- `SCIP solver not found` → Ensure `scip` is in the conda environment; check `conda list | grep scip`

Generating Analysis and Reports

Post-processing and insights:

```
cd personal_analysis
python run_summary.py           # Generate summary statistics and
                                tables from results
python interpret_results.py     # Analyze network results (flows,
                                LMPs, congestion)
python explore_scenarios.py     # Discover and list all available
                                scenario configurations
```

What each script produces:

- `run_summary.py` - Creates `summary_*.csv` with cost, ENS, shedding, and import metrics
- `interpret_results.py` - Generates flow analysis, LMP heatmaps, and congestion reports
- `explore_scenarios.py` - Lists scenario templates, year options, and cutout availability

Typical workflow:

1. Run scenarios via the web dashboard
2. Verify results appear in the Results tab with all 7 required CSVs
3. Run analysis scripts to generate additional insights
4. Use outputs for reporting or further investigation

Viewing and Analyzing Results

Primary method: Web Dashboard (recommended)

```
cd vizualizer && npm run dev           # http://localhost:3000
```

Features: scenario builder, live job monitoring, results browser with CSV previews and figures, live log tailing.

Legacy Tkinter UI (stable fallback):

```
python personal_dashboard/visualize_scenarios_ui_v2.py
```

Use if the web dashboard has issues; features are mostly equivalent but less polished.

Manual inspection:

```
ls results/                                # List all result folders
# Results must contain all 7 CSVs to be valid (see "Result Contract" below)
```

Adding Support for a New Year (e.g., 2024)

The system auto-detects new year templates. No manual UI updates needed.

Steps:

1. Create `personal_docs/scenario_template_2024.yaml` (copy from 2023, update year references)

```
# At top of file, update:
cutout: europe-2024-sarah3-era5
# Update any references to 2023 → 2024 in comments
```

2. Verify cutout file exists: `data/cutout/archive/v0.8/europe-2024-sarah3-era5.nc`
 - If missing, run: `cd personal_data_download && python download_cutout.py`
3. No further changes needed; `resolve_template_path()` in `config_builder.py` auto-detects the new template
4. Restart the web dashboard; the year should appear in the Cutout Year dropdown

Verification:

```
cd personal_diagnostics
python check_romania.py # Should not raise errors about missing cutouts
```

Key Architectural Decisions

1. **Immutable Template Pattern:** Templates in `personal_docs/` are read-only; user configs generated to `config/adversarial/generated/`
2. **Year-Specific Templates:** Auto-select logic eliminates manual template management; fallback safety if template missing
3. **Web-First Dashboard:** `vizualizer/` (Next.js) is the primary UI; Tkinter UI retained as stable fallback
4. **Queue-Based Execution:** Async subprocess management prevents blocking; job state persisted across app restarts
5. **New-Format Results Only:** Results page only displays comparison outputs meeting CSV validation criteria (discards legacy formats)
6. **Dual Editing Modes:** Form + YAML allows both guided and power-user workflows updating same underlying config

7. **Stress Test Modular Design:** Shock logic isolated in `romania_winter_stress.py` for testability and reusability
8. **Conda Auto-Discovery:** Web dashboard detects the conda env at startup; override via `PLANUI_CONDA_ENV` if needed

Vizualizer Data Flow

```
vizualizer/ reads  → results/                (comparison outputs)
                  → config/adversarial/generated/ (scenario configs)
                  → personal_docs/             (templates, read-only)
vizualizer/ writes → config/adversarial/generated/ (newly generated configs)
                  → logs/planui-web/           (per-job log files)
                  → vizualizer/.data/           (job state JSON)
```

Testing and Validation

Automated Testing

Unit tests for stress logic:

```
pytest test/test_romania_winter_stress.py -v # Run with verbose output to see
each test
```

Tests cover shock application, constraint generation, and result validation.

Manual Pre-Run Validation Checklist

Before running large scenario suites, verify all prerequisites:

1. Check config validity:

```
python personal_diagnostics/check_romania.py
```

Should show no errors about YAML syntax, missing cutouts, or invalid settings.

2. Run a quick baseline test:

```
python personal_runners/run_baseline_only.bat
```

Should complete in 30-60 seconds; verifies solver and network loading.

3. Validate result structure:

```
python personal_diagnostics/check_csv.py
```

Run after a scenario completes; ensures all 7 required CSVs are present and valid.

4. Verify data sources:

```
python personal_diagnostics/check_url.py
```

Tests external API connectivity; if any fail, downloads may fail during future runs.

Typical troubleshooting flow:

Check fails → Review error message → Fix (e.g., download cutout) → Re-check → Proceed

Notes for Future Development

Planned Enhancements

Bilingual Dashboard Support

- Web dashboard (Next.js) will soon support English/Romanian toggle similar to legacy Tkinter UI
- Translations managed in `vizualizer/src/app/lib/i18n.ts` (structure mirrors `personal_dashboard/scenario_manager/i18n.py`)
- Expected in next release

Tkinter UI Consolidation

- Legacy Tkinter UI (`personal_dashboard/`) will be retired once web dashboard achieves feature parity
- Current status: web dashboard covers all core workflows; Tkinter remains as stable fallback

System Maintenance

Cutout and Data Management

- Weather data (ERA5 cutouts) is large (~2GB per year)
- `personal_data_download/download_cutout.py` caches to `data/cutout/archive/v0.8/` for offline use
- Zenodo datasets auto-cached during first scenario run
- Delete `data/cutout/archive/` to force re-download if corruption suspected

Template Versioning Strategy

- Keep all year-specific templates in sync with each other

- Document breaking changes in [personal_docs/TEMPLATE_ARCHITECTURE.md](#)
- When updating stress test defaults, update all templates (not just 2023)
- Use git history to track template evolution

Performance Tuning

- Network solve time scales with:
 - **Cluster count:** Higher = slower but more spatially refined. Start with 5-10 for testing.
 - **Temporal resolution:** Daily is much faster than hourly. Use for sensitivity studies.
 - **Solver options:** `solver_logfile=false` and `min_iterations=1` can speed up test runs.
- Typical baseline solve: 5-15 min (10 clusters, hourly). Stress solves usually $\pm 10\%$ due to shock magnitude.

Extensibility

Adding new stress types:

1. Define new shock parameters in template YAML under `stress_test`.
2. Implement shock application logic in `scripts/romania_winter_stress.py`
3. Add constraint generation in same script
4. Update form controls in `visualizer/src/app/components/ScenarioBuilder.tsx`
5. Add new result metrics to reporting script

Supporting new geographic regions:

1. Extend PyPSA-Eur config scope (e.g., EU-wide instead of Balkans)
2. Create region-specific template: `personal_docs/scenario_template_<region>.yaml`
3. Update shock logic to apply only to target countries
4. Add region-specific result visualizations
5. Document in `personal_docs/REGION_GUIDE.md`